Terraform Final Task.

1. **Create VPC**
2. **Create Internet gateway**
3. **Create Custom Route Table**
4. **Create Subnet**
5. **Associate subnet with Route Table**
6. **Create Security Group to allow port 22.80,443**
7. **Create a network interface with an ip in the subnet that was created in step 4**
8. **Assign an elastic IP to the network interface created in step 7**
9. **Create Ubuntu server and install/enable apache2**

Objective

To create AWS infrastructure using Terraform by creating a VPC, Internet Gateway, Route Table, Subnet, Security Group, Network Interface, Elastic IP, and Ubuntu EC2 instance. Apache2 was installed automatically on the EC2 instance using a user-data script.

Step 1: Created Project Directory

1. Created a folder named terraform-final-task on my Windows system.
2. Opened the folder in Visual Studio Code.
3. Created the following Terraform files:
    - main.tf
    - variables.tf
    - terraform.tfvars
    - outputs.tf

Step 2: Configured Terraform Provider and Variables

Configured the AWS provider in main.tf with the region us-east-1.

variables.tf

```
variable "ssh_cidr" {
 description = "Your public IP address with /32"
 type      = string
```

```
}


variable "key_pair_name" {
  description = "Existing EC2 key pair name"
  type        = string
}
```

terraform.tfvars

```
ssh_cidr     = "YOUR_PUBLIC_IP/32"
key_pair_name = "mytest_key"
```

- Used my public IP address with /32 for SSH access.
- Used my existing EC2 key pair named mytest_key.

Initialized the Terraform working directory:

```
terraform init
```

Step 3: Created VPC

Added the aws_vpc resource block to create a VPC.

Code:

```
resource "aws_vpc" "main" {
  cidr_block         = "10.0.0.0/16"
  enable_dns_support   = true
  enable_dns_hostnames = true


  tags = {
    Name = "Custom-VPC"
  }
}
```

- Configured the VPC CIDR block as 10.0.0.0/16.
- Enabled DNS support and DNS hostnames.
- Applied the Terraform configuration and verified successful creation.

Step 4: Created Internet Gateway

Added the aws_internet_gateway resource block and attached it to the previously created VPC.

Code:

```
resource "aws_internet_gateway" "igw" {
 vpc_id = aws_vpc.main.id


 tags = {
  Name = "Custom-IGW"
 }
}
```

- Attached the Internet Gateway to the VPC.
- Applied the configuration and verified successful creation.

Step 5: Created Custom Route Table

Added the aws_route_table resource block.

Code:

```
resource "aws_route_table" "public_rt" {
 vpc_id = aws_vpc.main.id


 route {
  cidr_block = "0.0.0.0/0"
  gateway_id = aws_internet_gateway.igw.id
 }


 tags = {
  Name = "Public-Route-Table"
 }
}
```

- Associated the route table with the VPC.
- Configured the default route 0.0.0.0/0.
- Used the Internet Gateway as the target.
- Applied and verified successful creation.

Step 6: Created Public Subnet

Added the aws_subnet resource block.

Code:

```
resource "aws_subnet" "public_subnet" {
 vpc_id              = aws_vpc.main.id
 cidr_block          = "10.0.1.0/24"
 availability_zone   = "us-east-1a"
 map_public_ip_on_launch = true


 tags = {
  Name = "Public-Subnet"
 }
}
```

- Created a subnet inside the VPC.
- Configured the CIDR block as 10.0.1.0/24.
- Selected Availability Zone us-east-1a.
- Enabled automatic public IP assignment.
- Applied and verified successful creation.

Step 7: Associated Subnet with Route Table

Added the aws_route_table_association resource block.

Code:

```
resource "aws_route_table_association" "public_assoc" {
 subnet_id     = aws_subnet.public_subnet.id
 route_table_id = aws_route_table.public_rt.id
```

}

- Associated the public subnet with the custom route table.
- Applied the configuration and verified successful association.

Step 8: Created Security Group

Added the aws_security_group resource block.

Code:

```
resource "aws_security_group" "web_sg" {
 name       = "Web-Security-Group"
 description = "Allow SSH, HTTP and HTTPS"
 vpc_id     = aws_vpc.main.id

 ingress {
  description = "SSH"
  from_port   = 22
  to_port     = 22
  protocol    = "tcp"
  cidr_blocks = [var.ssh_cidr]
 }

 ingress {
  description = "HTTP"
  from_port   = 80
  to_port     = 80
  protocol    = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
 }

 ingress {
```

```
  description = "HTTPS"
  from_port   = 443
  to_port     = 443
  protocol    = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
 }

 egress {
  from_port   = 0
  to_port     = 0
  protocol    = "-1"
  cidr_blocks = ["0.0.0.0/0"]
 }

 tags = {
  Name = "Web-Security-Group"
 }
}
```

Configured the inbound rules:

Port	Protocol	Purpose
22	TCP	SSH
80	TCP	HTTP
443	TCP	HTTPS

- Allowed SSH access from my public IP address.
- Allowed HTTP and HTTPS access from anywhere.
- Configured an outbound rule to allow all traffic.
- Applied and verified successful creation.

Step 9: Created Network Interface

Added the aws_network_interface resource block.

Code:

```
resource "aws_network_interface" "web_eni" {
 subnet_id      = aws_subnet.public_subnet.id
 private_ips    = ["10.0.1.10"]
 security_groups = [aws_security_group.web_sg.id]


 tags = {
  Name = "Web-Network-Interface"
 }
}
```

- Associated the Network Interface with the public subnet.
- Assigned the private IP address 10.0.1.10.
- Attached the previously created Security Group.
- Applied and verified successful creation.

Step 10: Assigned Elastic IP

Added the aws_eip resource block.

Code:

```
resource "aws_eip" "web_eip" {
 domain                = "vpc"
 network_interface       = aws_network_interface.web_eni.id
 associate_with_private_ip = "10.0.1.10"


 depends_on = [
  aws_internet_gateway.igw
 ]
```

```
 tags = {
  Name = "Web-Elastic-IP"
 }
}
```

- Allocated an Elastic IP for VPC usage.
- Associated it with the created Network Interface.
- Specified the private IP address 10.0.1.10.
- Added a dependency on the Internet Gateway.
- Applied and verified successful allocation and association.

Step 11: Retrieved Ubuntu AMI

Added an AWS AMI data source to retrieve the latest Ubuntu 24.04 LTS image.

Code:

```
data "aws_ami" "ubuntu" {
 most_recent = true
 owners      = ["099720109477"]

 filter {
  name   = "name"
  values = ["ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-server-*"]
 }

 filter {
  name   = "architecture"
  values = ["x86_64"]
 }

 filter {
  name   = "virtualization-type"
```

```
  values = ["hvm"]
 }
}
```

- Used the official Ubuntu owner ID 099720109477.
- Selected the latest matching Ubuntu 24.04 LTS AMI.
- Filtered for the x86_64 architecture and HVM virtualization type.

Step 12: Created Ubuntu EC2 Instance and Installed Apache2

Added the aws_instance resource block.

Code:

```
resource "aws_instance" "ubuntu_server" {
 ami           = data.aws_ami.ubuntu.id
 instance_type = "t3.micro"
 key_name      = var.key_pair_name

 network_interface {
  network_interface_id = aws_network_interface.web_eni.id
  device_index         = 0
 }

 user_data = <<-EOF
  #!/bin/bash
  apt-get update -y
  apt-get install apache2 -y
  systemctl enable apache2
  systemctl start apache2
 EOF

 tags = {
```

```
  Name = "Ubuntu-Web-Server"
 }


 depends_on = [
  aws_eip.web_eip,
  aws_route_table_association.public_assoc
 ]
}
```

Configured the EC2 instance with:

- AMI: Ubuntu 24.04 LTS
- Instance type: t3.micro
- Key pair: mytest_key
- Network Interface: Previously created Network Interface

Used the Terraform user_data argument to install and start Apache2 automatically.

The user-data script performed the following operations:

1. Updated Ubuntu packages.
2. Installed Apache2.
3. Enabled Apache2 to start automatically during system boot.
4. Started the Apache2 service.

Applied the configuration and verified successful EC2 creation.

Step 13: Configured Terraform Outputs

Added the following output blocks in outputs.tf.

Code:

```
output "vpc_id" {
 value = aws_vpc.main.id
}


output "elastic_ip" {
```

```
  value = aws_eip.web_eip.public_ip
}

output "private_ip" {
  value = aws_network_interface.web_eni.private_ip
}

output "instance_id" {
  value = aws_instance.ubuntu_server.id
}
```

These outputs were configured to display the VPC ID, Elastic IP, private IP, and EC2 instance ID.

Step 14: Verified Terraform Resources

Executed the following command:

terraform state list

Verified that all nine AWS resources were present in the Terraform state:

aws_vpc.main

aws_internet_gateway.igw

aws_route_table.public_rt

aws_subnet.public_subnet

aws_route_table_association.public_assoc

aws_security_group.web_sg

aws_network_interface.web_eni

aws_eip.web_eip

aws_instance.ubuntu_server

The Ubuntu AMI data source was also present in the state.

Confirmed that Terraform successfully created all nine AWS resources.

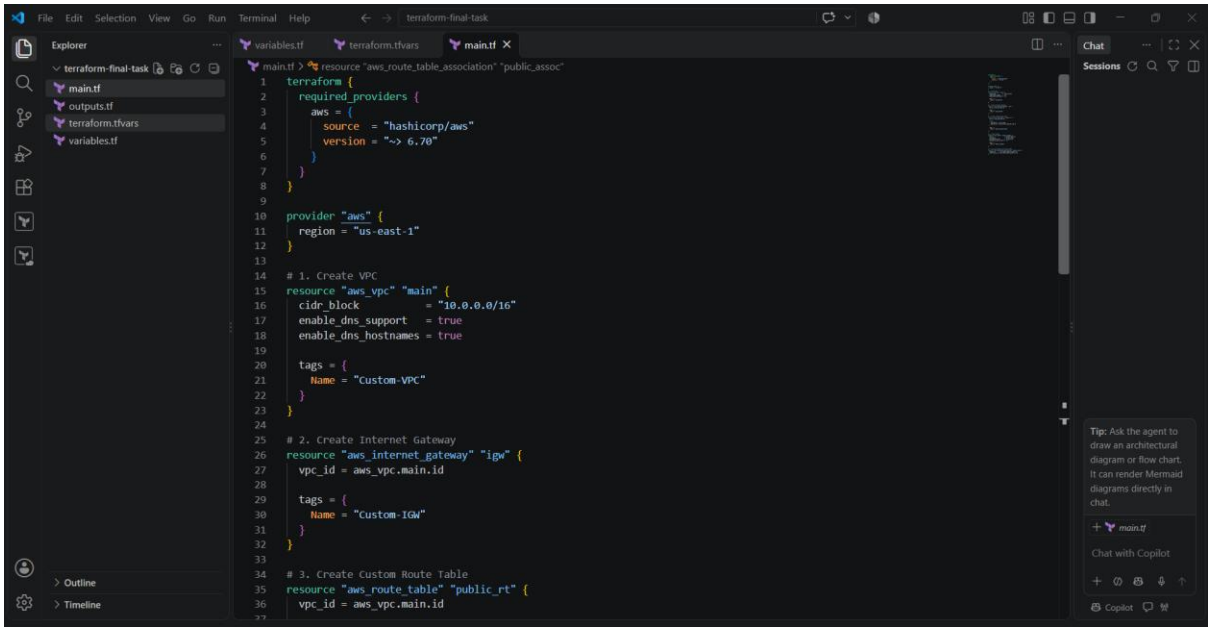
Step 15: Verified Apache2 Web Server

1. Retrieved the Elastic IP address associated with the Network Interface.
2. Waited for the EC2 user-data script to finish installing Apache2.
3. Opened the Elastic IP address in a web browser using HTTP.
4. Successfully accessed the Apache2 Ubuntu Default Welcome Page.
5. Confirmed that the Apache HTTP server was installed, running, and accessible over the internet.

Final Result

- Successfully created the VPC infrastructure using Terraform.
- Created all nine AWS resources.
- Configured the Security Group with SSH, HTTP, and HTTPS rules.
- Created and configured the Network Interface and Elastic IP.
- Launched an Ubuntu EC2 instance.
- Installed and started Apache2 automatically using Terraform user data.
- Successfully accessed the Apache2 default welcome page through the Elastic IP address.

Status: Final Terraform Task Completed Successfully.



variables.tf | terraform.tfvars | main.tf | outputs.tf

terraform.tfvars > key_pair_name

```
ssh_cidr      = "49.43.231.5/32"
key_pair_name = "mytest_key"
```

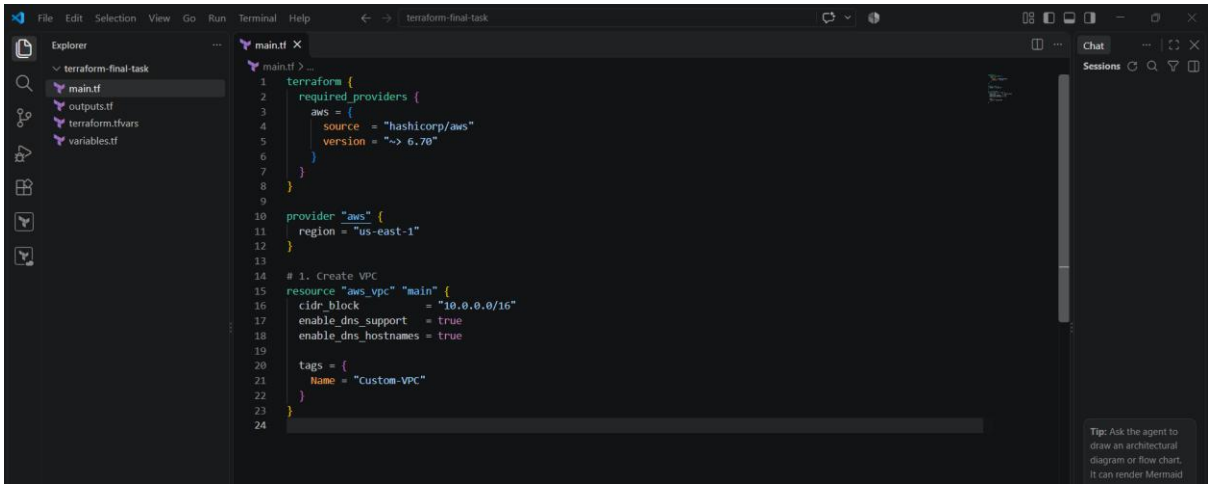
variables.tf | terraform.tfvars | main.tf | outputs.tf

variables.tf > variable "key_pair_name"

```
variable "ssh_cidr" {
  description = "Your public IP address with /32"
  type        = string
}

variable "key_pair_name" {
  description = "Existing EC2 key pair name"
  type        = string
}
```

1. Create VPC



```
Problems  Output  Debug Console  Terminal  Ports                                    powershell

PS C:\terraform-final-task> terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 6.70"...
- Installing hashicorp/aws v6.71.0...
- Installed hashicorp/aws v6.71.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
```

```
Problems  Output  Debug Console  Terminal  Ports                                    powershell

PS C:\terraform-final-task> terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_vpc.main will be created
  + resource "aws_vpc" "main" {
      + arn                              = (known after apply)
      + cidr_block                       = "10.0.0.0/16"
      + default_network_acl_id           = (known after apply)
      + default_route_table_id           = (known after apply)
      + default_security_group_id        = (known after apply)
      + dhcp_options_id                  = (known after apply)
      + enable_dns_hostnames             = true
      + enable_dns_support               = true
```

```
Problems  Output  Debug Console  Terminal  Ports                                    powershell

      + "Name" = "Custom-VPC"
    }
}

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_vpc.main: Creating...
aws_vpc.main: Creation complete after 5s [id=vpc-02cadf2f664be828e]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS C:\terraform-final-task>
```

Your VPCs

VPCs | VPC encryption controls

Your VPCs (2) Q help

Last updated less than a minute ago | Actions | Create VPC

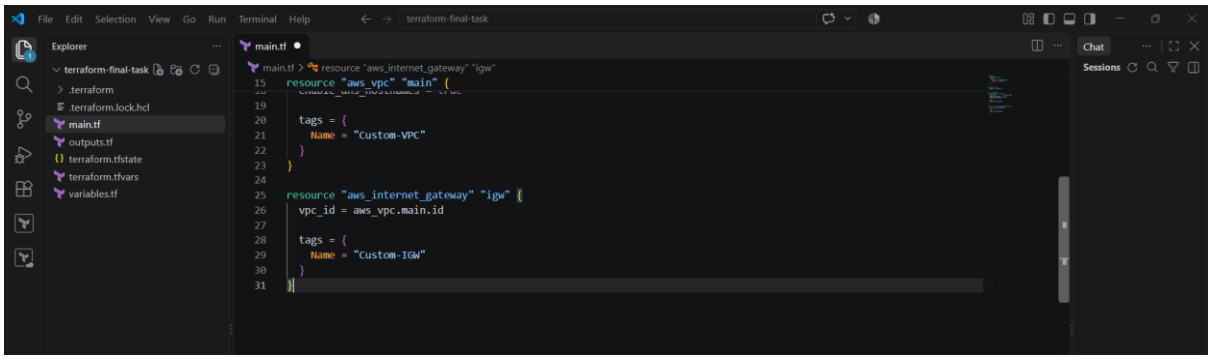
Filter VPCs by attribute or tag

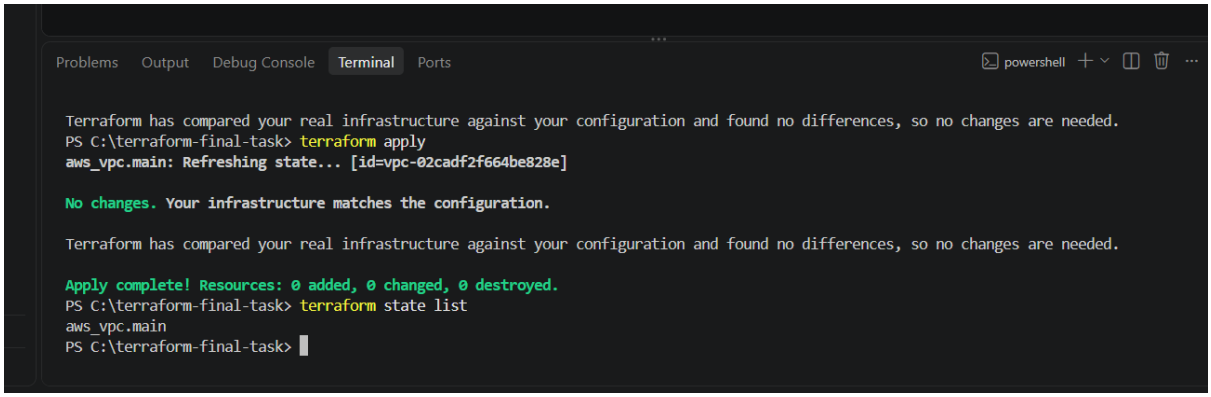
1

Name	VPC ID	State	Encryption c...	Encrypt
Custom-VPC	vpc-02cadf2f664be828e	Available	–	–
–	vpc-033d33753899582ee	Available	–	–

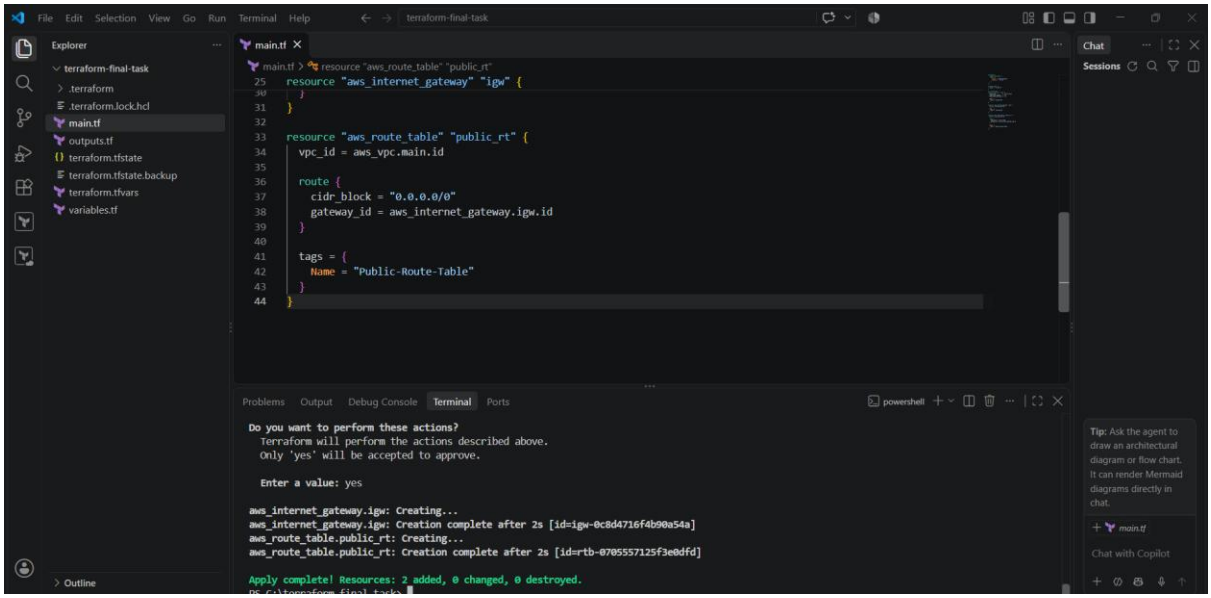
Select a VPC above

2. Create Internet gateway





3. Create Custom Route Table



```
Problems  Output  Debug Console  Terminal  Ports                    powershell

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_internet_gateway.igw: Creating...
aws_internet_gateway.igw: Creation complete after 2s [id=igw-0c8d4716f4b90a54a]
aws_route_table.public_rt: Creating...
aws_route_table.public_rt: Creation complete after 2s [id=rtb-0705557125f3e0dfd]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
PS C:\terraform-final-task>
```

```
Problems  Output  Debug Console  Terminal  Ports                    powershell

  Enter a value: yes

aws_internet_gateway.igw: Creating...
aws_internet_gateway.igw: Creation complete after 2s [id=igw-0c8d4716f4b90a54a]
aws_route_table.public_rt: Creating...
aws_route_table.public_rt: Creation complete after 2s [id=rtb-0705557125f3e0dfd]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
PS C:\terraform-final-task> terraform state list
aws_internet_gateway.igw
aws_route_table.public_rt
aws_vpc.main
PS C:\terraform-final-task>
```

Internet gateways (2) Q help

Actions | Create internet gateway

Find internet gateways by attribute or tag

1

Name	Internet gateway ID	State	VPC ID
–	igw-00d4b8aeeac58e0fd	Attached	vpc-033d337538995
Custom-IGW	igw-0c8d4716f4b90a54a	Attached	vpc-02cadf2f664be8

Route tables (3) Q help

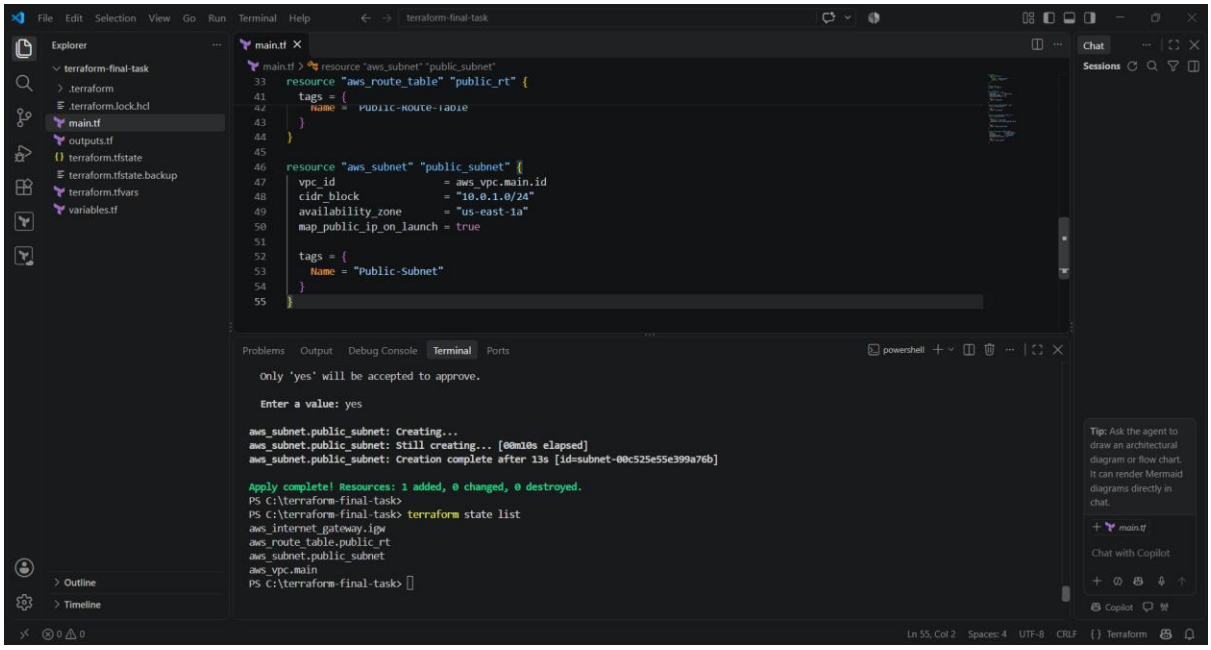
Last updated less than a minute ago | Actions | Create route table

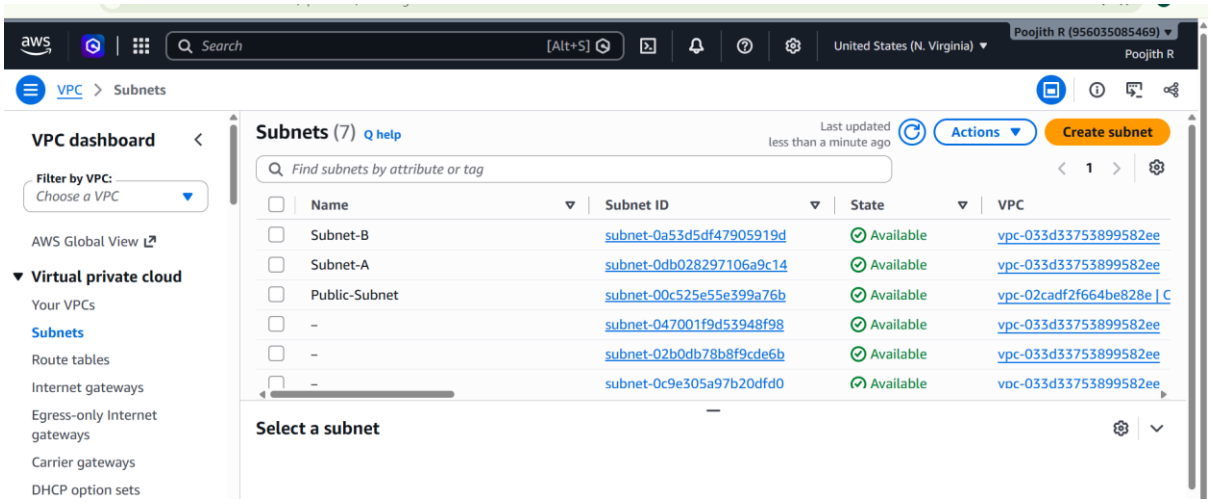
Find route tables by attribute or tag

1

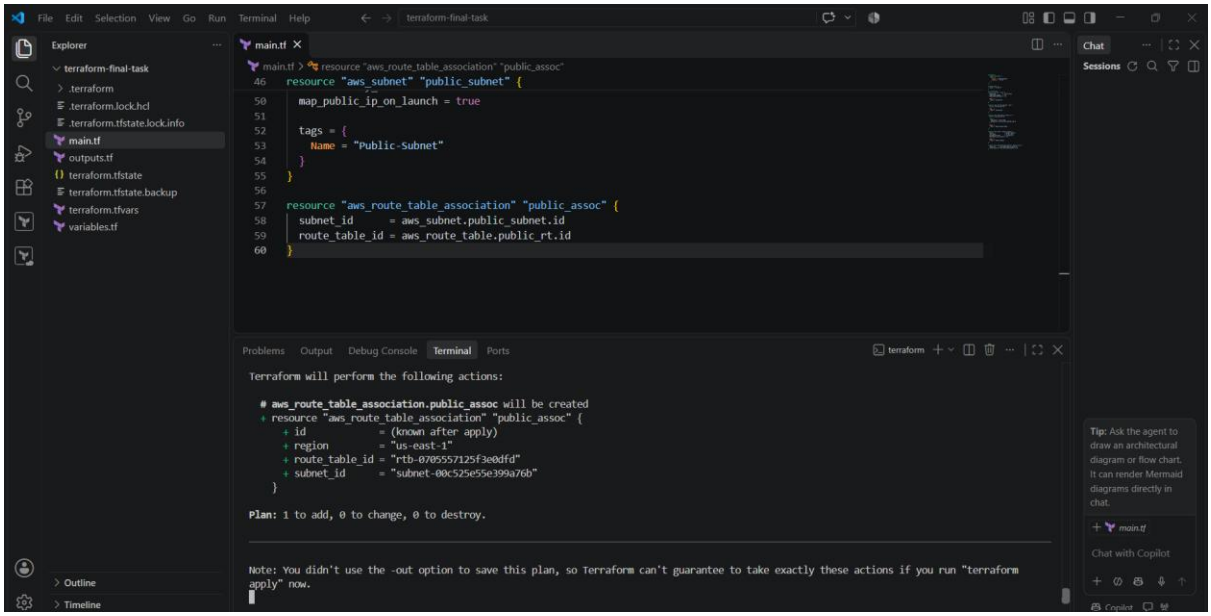
Name	Route table ID	Explicit subnet associations	Edge associations
Public-Route-Table	rtb-0705557125f3e0dfd	–	–
–	rtb-0427a85de810e3bee	–	–
–	rtb-0e402ae6c52cb9c61	subnet-0db028297106a9c14 / ...	–

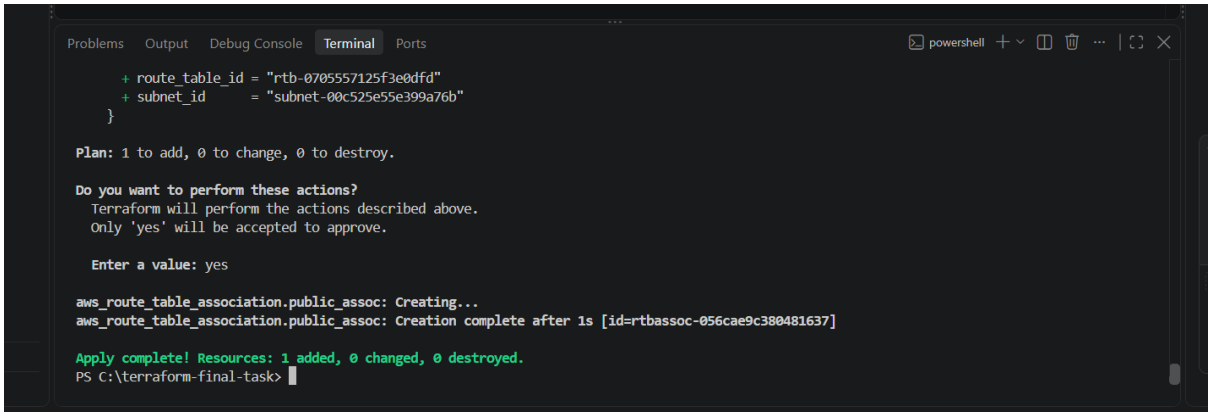
4. Create Subnet

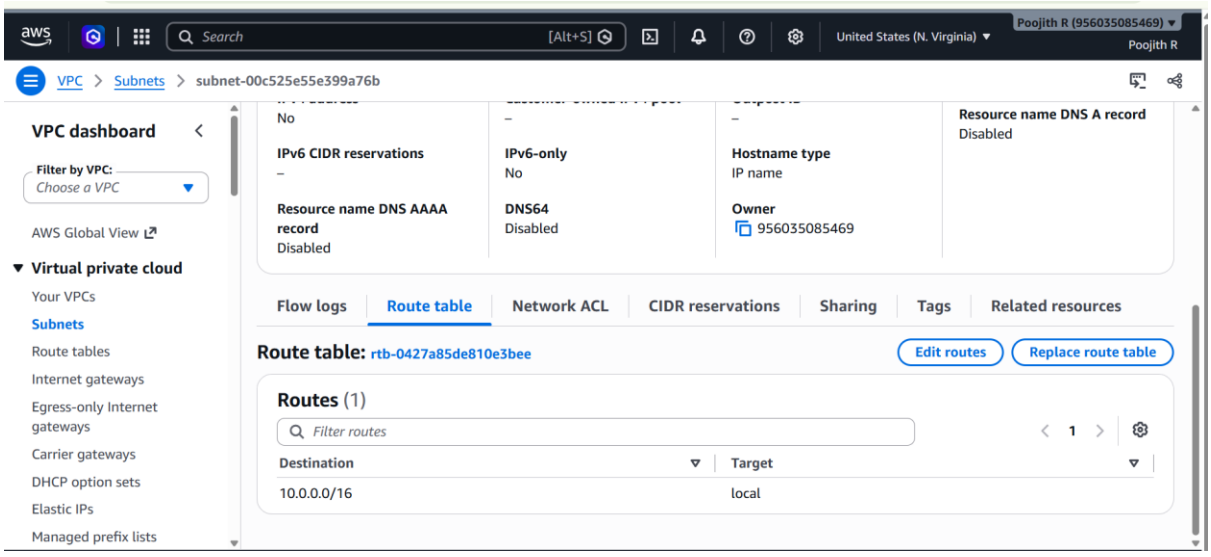




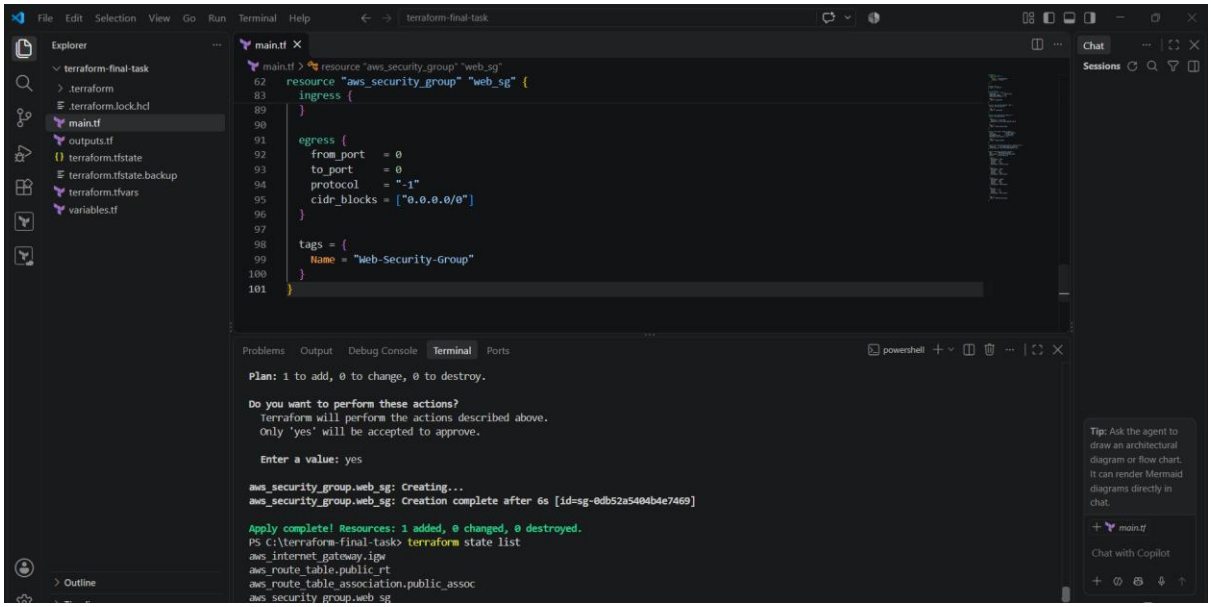
5. Associate subnet with Route Table

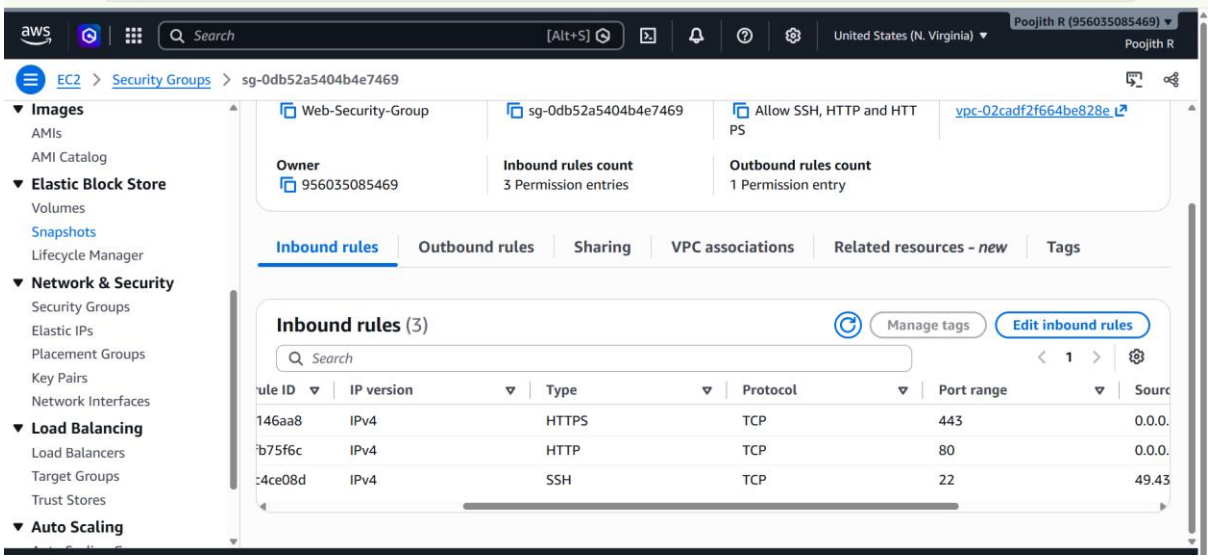




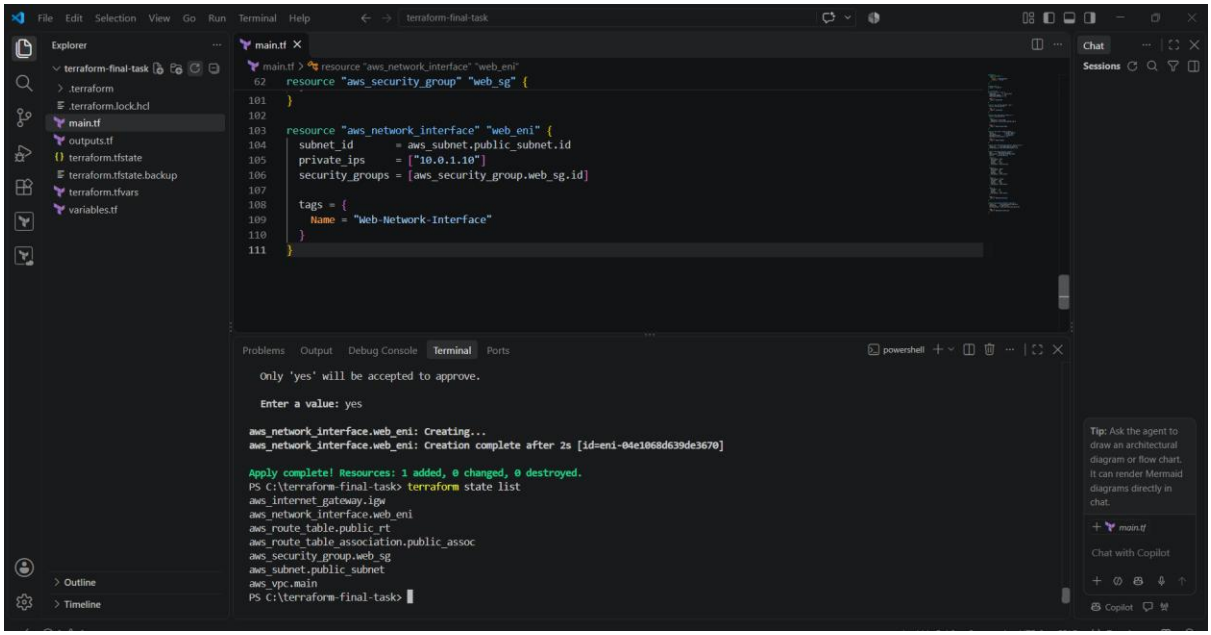


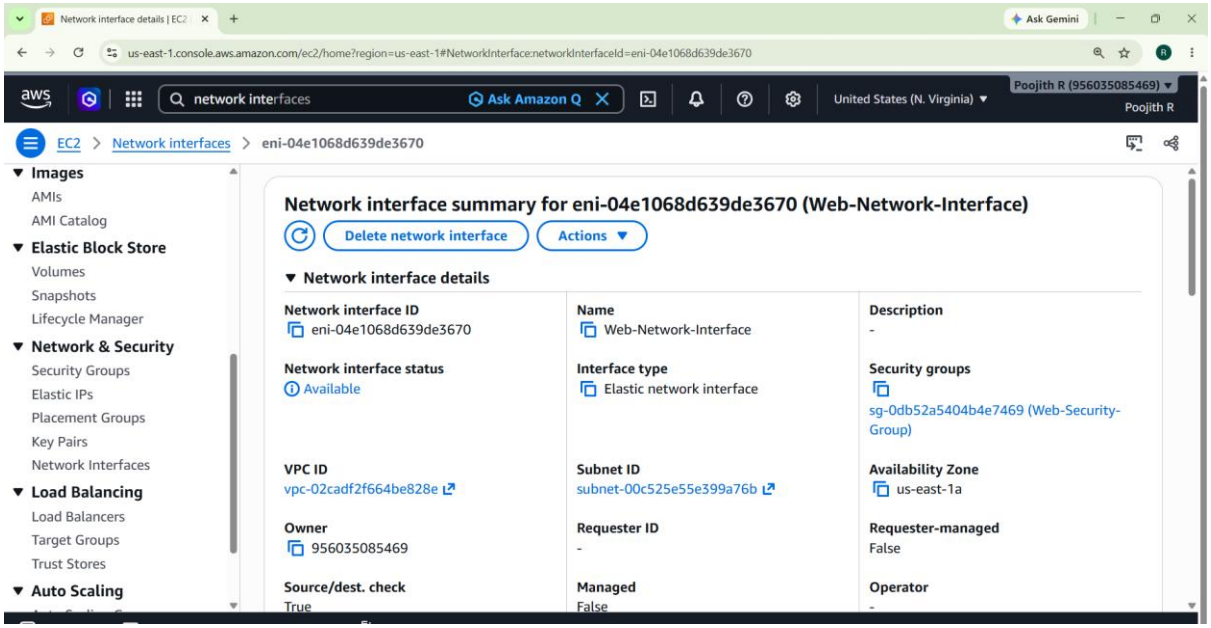
6. Create Security Group to allow port 22.80,443



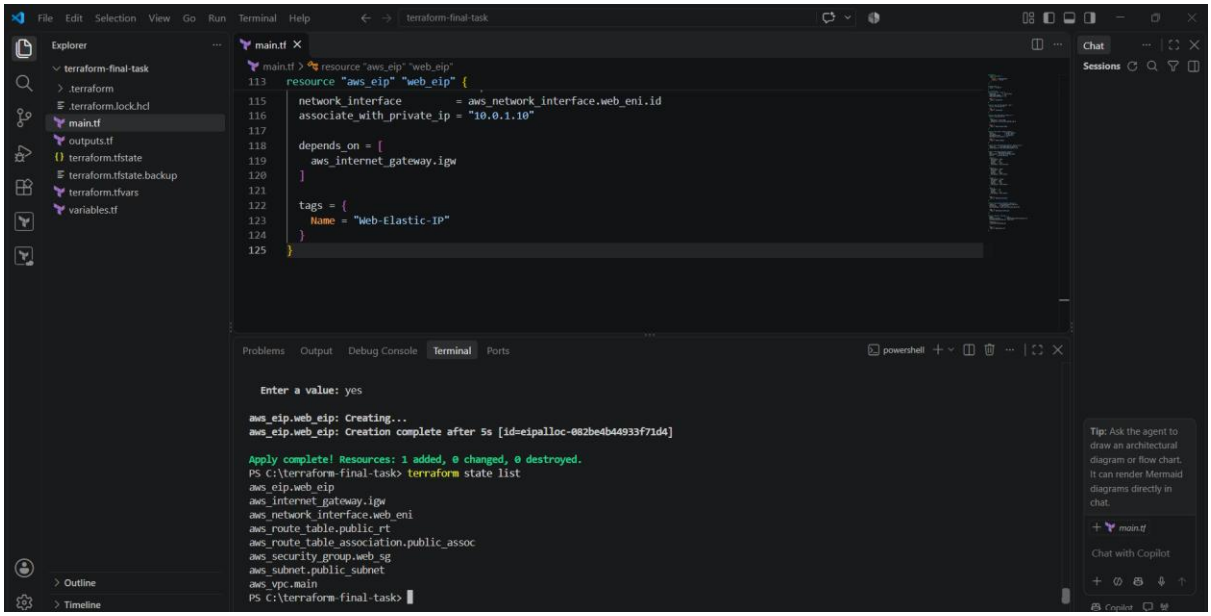


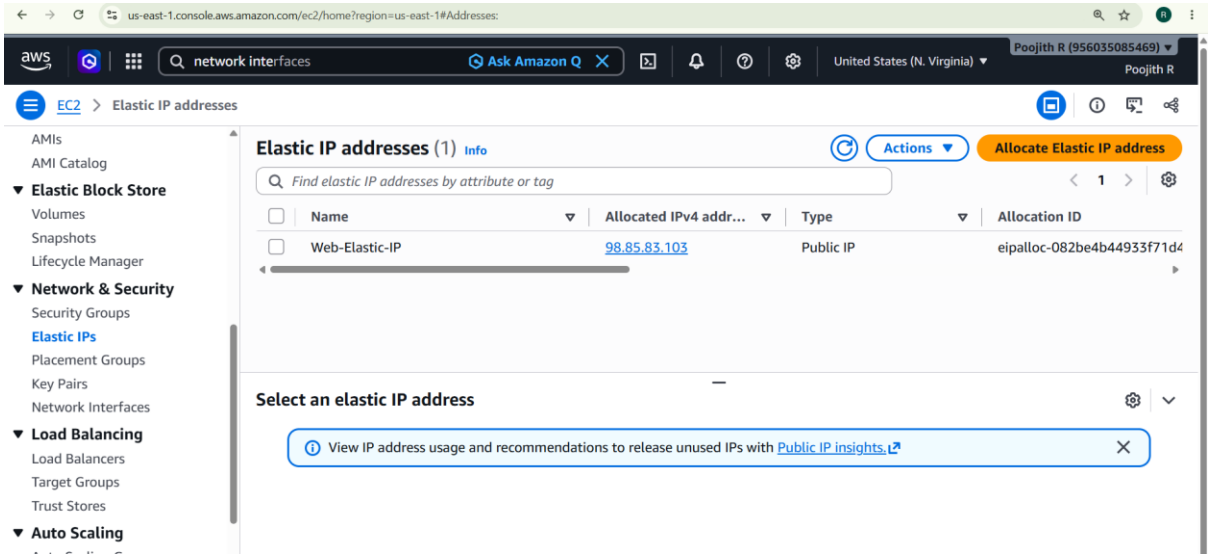
7. Create a network interface with an ip in the subnet that was created in step 4



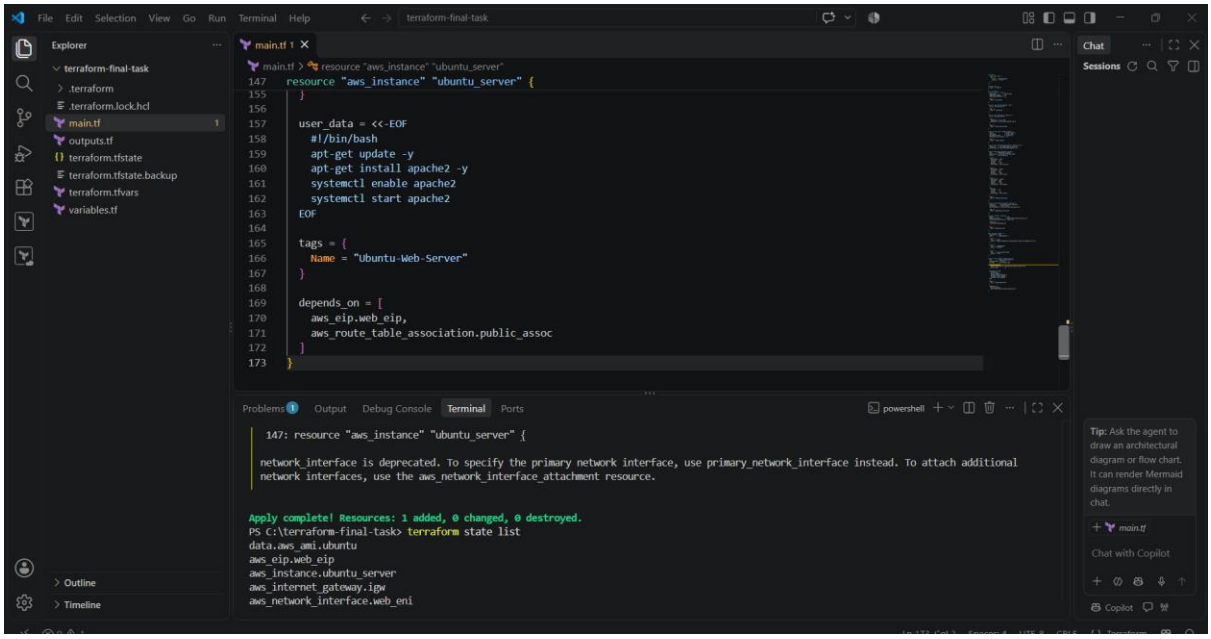


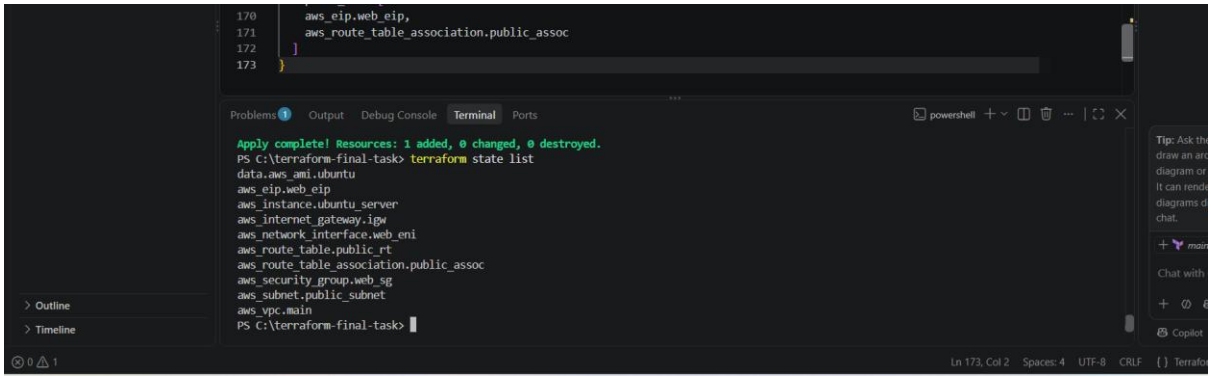
8. Assign an elastic IP to the network interface created in step 7

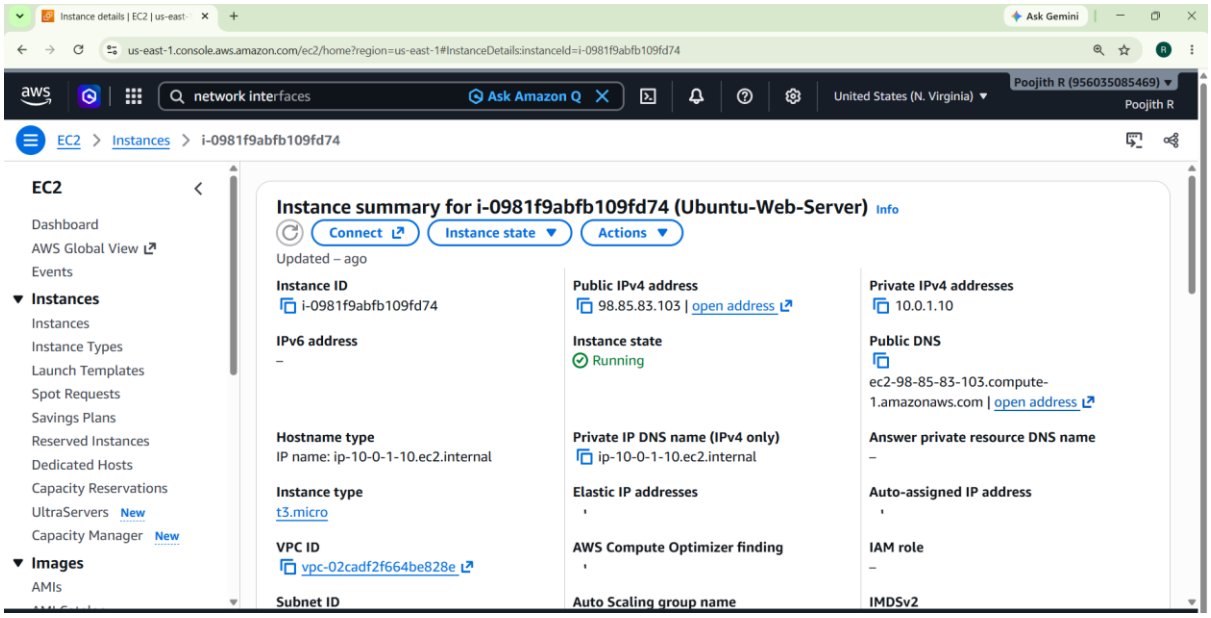


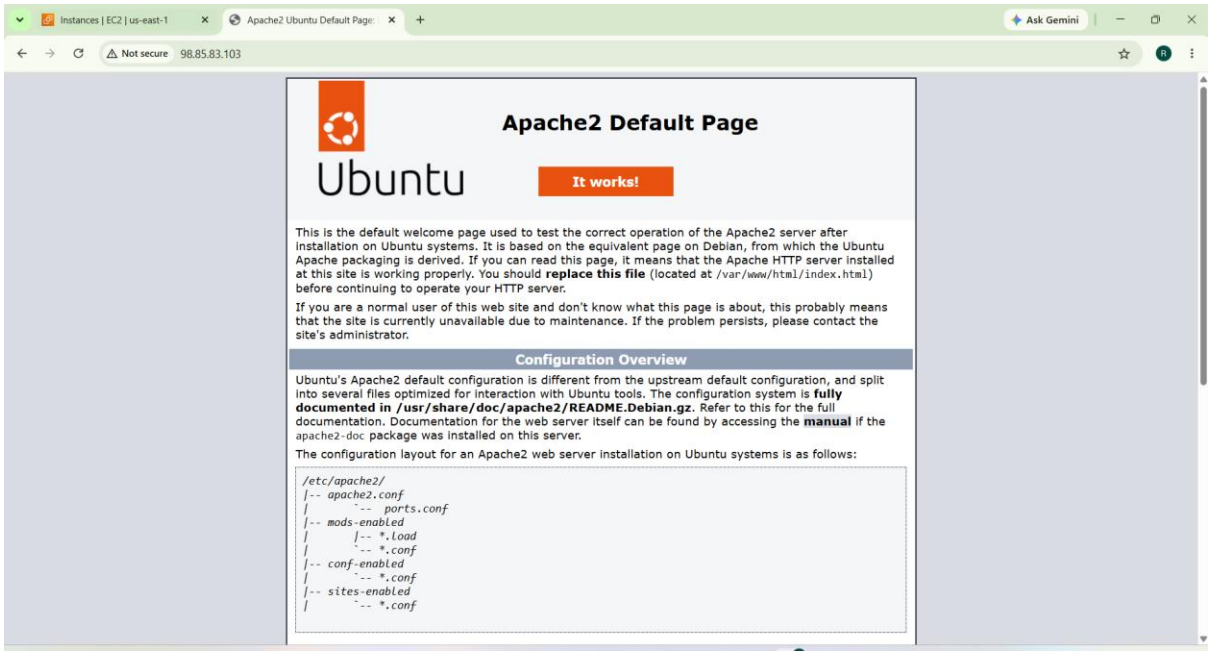


9. Create Ubuntu server and install/enable apache2


main.tf
resource "aws_instance" "ubuntu_server" {
}
user_data = <<-EOF
#!/bin/bash
apt-get update -y
apt-get install apache2 -y
systemctl enable apache2
systemctl start apache2
EOF
tags = {
Name = "Ubuntu-Web-Server"
}
depends_on = [
aws_eip.web_eip,
aws_route_table_association.public_assoc
]
}
Problems Output Debug Console Terminal Ports
powershell
147: resource "aws_instance" "ubuntu_server" {
network_interface is deprecated. To specify the primary network interface, use primary_network_interface instead. To attach additional network interfaces, use the aws_network_interface_attachment resource.
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS C:\terraform-final-task> terraform state list
data.aws_ami.ubuntu
aws_eip.web_eip
aws_instance.ubuntu_server
aws_internet_gateway.igw
aws_network_interface.web_eni


aws_eip.web_eip,
aws_route_table_association.public_assoc
]
}
Problems Output Debug Console Terminal Ports
powershell
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS C:\terraform-final-task> terraform state list
data.aws_ami.ubuntu
aws_eip.web_eip
aws_instance.ubuntu_server
aws_internet_gateway.igw
aws_network_interface.web_eni
aws_route_table.public_rt
aws_route_table_association.public_assoc
aws_security_group.web_sg
aws_subnet.public_subnet
aws_vpc.main
PS C:\terraform-final-task>
Outline
Timeline


us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#InstanceDetails:instanceId=i-0981f9abfb109fd74
network interfaces
Ask Amazon Q
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
EC2 > Instances > i-0981f9abfb109fd74
EC2
Dashboard
AWS Global View
Events
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
UltraServers New
Capacity Manager New
Images
AMIs
Instance summary for i-0981f9abfb109fd74 (Ubuntu-Web-Server) Info
Connect
Instance state
Actions
Updated – ago
Instance ID
i-0981f9abfb109fd74
IPv6 address
–
Hostname type
IP name: ip-10-0-1-10.ec2.internal
Instance type
t3.micro
VPC ID
vpc-02cadf2f664be828e
Subnet ID
Public IPv4 address
98.85.83.103 | open address
Instance state
Running
Private IP DNS name (IPv4 only)
ip-10-0-1-10.ec2.internal
Elastic IP addresses
Auto Scaling group name
Private IPv4 addresses
10.0.1.10
Public DNS
ec2-98-85-83-103.compute-1.amazonaws.com | open address
Answer private resource DNS name
–
Auto-assigned IP address
IAM role
–
IMDSv2


Instances | EC2 | us-east-1
Apache2 Ubuntu Default Page:
Ask Gemini
Not secure
98.85.83.103
Apache2 Default Page
Ubuntu
It works!
This is the default welcome page used to test the correct operation of the Apache2 server after installation on Ubuntu systems. It is based on the equivalent page on Debian, from which the Ubuntu Apache packaging is derived. If you can read this page, it means that the Apache HTTP server installed at this site is working properly. You should replace this file (located at /var/www/html/index.html) before continuing to operate your HTTP server.
If you are a normal user of this web site and don't know what this page is about, this probably means that the site is currently unavailable due to maintenance. If the problem persists, please contact the site's administrator.
Configuration Overview
Ubuntu's Apache2 default configuration is different from the upstream default configuration, and split into several files optimized for interaction with Ubuntu tools. The configuration system is fully documented in /usr/share/doc/apache2/README.Debian.gz. Refer to this for the full documentation. Documentation for the web server itself can be found by accessing the manual if the apache2-doc package was installed on this server.
The configuration layout for an Apache2 web server installation on Ubuntu systems is as follows:
/etc/apache2/
|-- apache2.conf
| `-- ports.conf
|-- mods-enabled
| |-- *.load
| `-- *.conf
|-- conf-enabled
| `-- *.conf
|-- sites-enabled
| `-- *.conf